## Question #21 (Topic: Exam A) *Topic 1*

Your document extraction tool uses ML models to extract invoice fields (vendor, amount, date). The models return confidence scores (0.0-1.0) for each extracted field. In production, you observe: (1) the agent proceeds with low-confidence extractions that are incorrect 23% of the time, and (2) the agent requests unnecessary human review for 31% of extractions that were actually correct. How should you restructure the tool's output?

A. Return fields with their raw confidence scores and add detailed few-shot examples to your system prompt demonstrating how to interpret different confidence ranges and when to request human review.

B. Return fields with confidence scores, plus a request_review boolean computed using your tested confidence thresholds, along with a review_reasons array explaining which fields triggered review. **Most Voted**

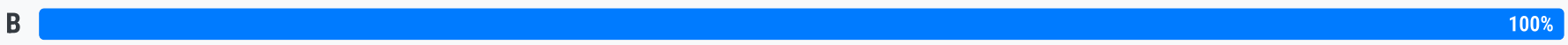
C. Compute an aggregate extraction_quality score across all fields and return it alongside the extracted values. Include a text summary describing the overall extraction reliability.

D. Return fields organized into verified and needs_verification objects based on confidence thresholds.

**Correct Answer: B**

Community vote currently supports Answer B

**Community vote distribution**




## Question #22 (Topic: Exam A) *Topic 1*

Your product search tool queries an external catalog API and returns matching items. In production, you observe the agent frequently retries searches immediately after receiving zero results, treating "no matches found" as a failure requiring retry. The external API returns HTTP 200 with an empty results array – a valid response. How should you restructure the tool's result to help the agent correctly interpret empty result sets?

A. Add a suggestions field containing alternative search strategies when results are empty, helping guide the agent toward more productive follow-up queries.

B. Return a natural language string describing the outcome, allowing the agent to interpret the result contextually based on the message content.

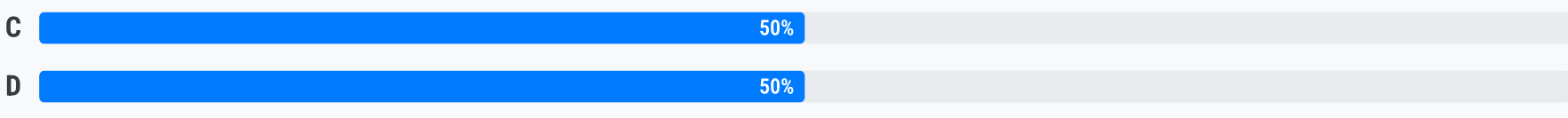
C. Return a result object with isError: true and a message explaining no products matched. **Most Voted**

D. Return a structured result with a success boolean and results array, reserving isError: true for actual execution failures only.

**Correct Answer: D**

Community vote currently supports Answer C

**Community vote distribution**



**Question #23** (Topic: Exam A) *Topic 1*

Your MCP server includes archive_file(file_id) and delete_file(file_id) tools. Production logs show the agent calls delete_file when users ask to "remove old backups," but company policy requires archiving backup files. Both tools currently have minimal descriptions: "Archives a file" and "Deletes a file." Which change most directly improves tool selection for this scenario?

A. Expand tool descriptions to clarify use cases, adding guidance like "Do not use for backup files" to delete_file. **Most Voted**

B. Add a confirmation step that requires users to type "CONFIRM DELETE" before delete_file executes.

C. Implement server-side validation that rejects delete_file calls for files tagged as backups, returning an error message suggesting archive_file.

D. Add few-shot examples to the system prompt demonstrating that requests involving "backup" or "old" should use archive_file.

**Correct Answer: A**

Community vote currently supports Answer A

**Community vote distribution**




**Question #24** (Topic: Exam A) *Topic 1*

Your track_shipment(tracking_id) tool queries an external logistics API that sometimes fails – the API may be temporarily unavailable, the tracking ID may be malformed, or the shipment may not exist. Currently, your tool raises a Python exception when errors occur. Users report the agent gives unhelpful responses like "I'm having trouble with that request" instead of suggesting alternatives such as verifying the tracking number format or checking by order number. How should you handle errors in tool results?

A. Return a generic error response (e.g., {"success": false, "error": "lookup_failed"}) for all failure cases to maintain a consistent schema and avoid exposing internal error details.

B. Return structured error information as normal tool output including error type, recoverability status, and actionable context for the user. **Most Voted**

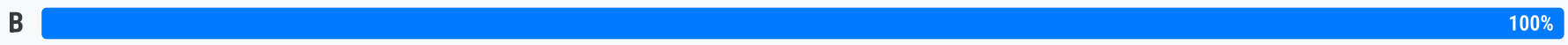
C. Create dedicated error-recovery tools (retry_tracking_lookup, search_by_order_number) that the model can invoke after the primary tracking tool returns a failure indicator.

D. Implement retry logic with exponential backoff inside the tool implementation so transient errors are automatically handled and only return a result after all retry attempts are exhausted.

**Correct Answer: B**

Community vote currently supports Answer B

**Community vote distribution**



**Question #25** (Topic: Exam A) *Topic 1*

Your MCP server implements a check_availability tool that queries an external calendar API. During testing, you encounter three error conditions: (1) the tool is called with a malformed request missing the required user_email parameter, (2) the calendar API returns a 404 because the specified user doesn't exist in the calendar system, and (3) the calendar API returns a 503 because the service is temporarily unavailable. How should each error be reported according to MCP's error handling design?

A. Report all three as tool results with isError: true. **Most Voted**

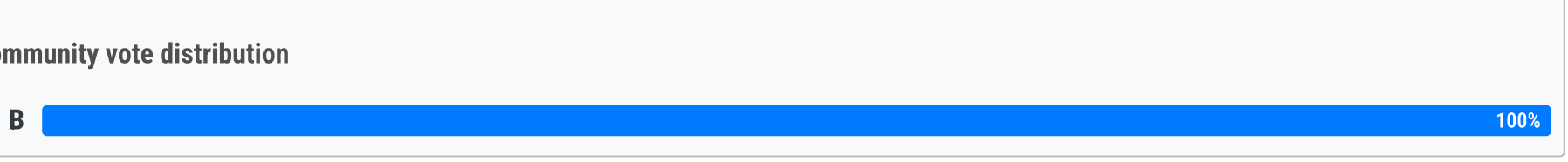
B. Report errors 1 and 2 as JSON-RPC protocol errors; report error 3 as a tool result with isError: true.

C. Report error 1 as a JSON-RPC protocol error; report errors 2 and 3 as tool results with isError: true.

D. Report all three as JSON-RPC protocol errors.

**Correct Answer: C** Community vote currently supports Answer A

**Community vote distribution**




**Question #26** (Topic: Exam A) *Topic 1*

Your send_notification tool calls third-party messaging APIs. When these services time out during delivery, you cannot determine whether the message was actually sent. Currently, the tool returns is_error: true with a generic "Notification failed" message for all timeouts. Production monitoring reveals agents automatically retry these failures, frequently causing users to receive duplicate notifications. How should you modify the error response?

A. Return is_error: true with a structured field retry_safe: true for timeouts, distinguishing them from permanent failures that should not be retried.

B. Return is_error: true with a message communicating uncertainty: "Timeout – status unknown. Message may have been sent. Avoid retry." **Most Voted**

C. Return is_error: true with a message encouraging retry: "Delivery service temporarily unavailable. Please retry the notification."

D. Return is_error: true with the original message content echoed back.

**Correct Answer: B** Community vote currently supports Answer B

**Community vote distribution**

**Question #27** (Topic: Exam A) *Topic 1*

Your control_device tool manages smart home devices through external APIs. When a device doesn't respond within the timeout period, the tool returns an error. Production logs show that the agent simply tells users "the device is not responding" without offering helpful next steps. Which error response structure would best enable the agent to provide useful follow-up?

A. Set is_error: true with a structured technical error containing the device ID, timeout duration, and raw API response code for debugging purposes.

B. Set is_error: true with a brief "Device offline" message and provide a separate tool the agent can call to retrieve context-specific troubleshooting suggestions.

C. Set is_error: false with an optimistic message indicating the command was dispatched successfully but device acknowledgment is still pending.

D. Set is_error: true with a message explaining the likely cause and suggesting troubleshooting steps the agent can offer the user.

**Correct Answer: D**

**Question #28** (Topic: Exam A) *Topic 1*

Your expense reimbursement agent processes employee requests using a process_reimbursement tool. Company policy requires that reimbursements above $500 must be approved by a manager before funds are disbursed. The agent handles hundreds of requests daily, and you need the threshold enforcement to be tamper-proof regardless of how the agent is prompted. Which design ensures the $500 approval threshold cannot be bypassed?

A. The process_reimbursement tool accepts amount and details, and internally enforces the threshold: amounts <$500 are auto-disbursed and the tool returns a success confirmation; amounts >$500 cause the tool to create a pending approval request and return a status indicating manager review is pending.

B. The process_reimbursement tool accepts an approved_by_manager: boolean parameter. The system prompt instructs the agent to only set this to true after confirming that a manager has approved the request. A nightly audit script reviews all reimbursements where approved_by_manager was set to true.

C. Provide two tools: auto_reimburse (hard-coded limit of $500) and request_manager_approval. Include detailed system prompt instructions telling the agent to check the amount and call the appropriate tool. Add a PostToolUse hook that logs which tool was called for auditing.

D. Implement the threshold check in a PreToolUse hook that inspects the amount parameter before process_reimbursement executes. If the amount exceeds $500, the hook modifies the tool call to add a requires_approval: true flag, which the tool checks before disbursing.

**Correct Answer: A**

**Question #29** (Topic: Exam A) *Topic 1*

Your content curation agent discovers articles, analyzes each for relevance, then adds selected articles to themed collections. With separate discover_articles(topic), analyze_article(id), and add_to_collection(article_id, collection_id) tools, you observe 18+ sequential tool calls per request, causing latency issues. The agent must make editorial judgments about which articles fit a collection's theme – this requires seeing all candidates with their analysis scores simultaneously to select a cohesive set. What tool composition best addresses efficiency while preserving editorial judgment?

A. Keep all tools separate but implement response caching for analyze_article calls.

B. Create a curate_collection(topic, collection_id) tool that handles discovery, analysis, and selection internally using configurable quality thresholds.

C. Create a discover_and_analyze(topic) composite tool that returns all candidates with their analysis scores, keeping add_to_collection separate for selective calls.

D. Add a preview_curation(topic, collection_id) tool that shows what would be added based on predefined rules, with an approve_curation() tool to confirm.

**Correct Answer: C**

**Question #30** (Topic: Exam A) *Topic 1*

The document analysis agent has a single analyze_documnet tool that takes a document and a free-text instruction parameter. During evaluation, requests like "extract the key financial metrics" often return narrative summaries, while "summarize the methodology" sometimes returns raw data tables. The synthesis agent reports that 35% of analysis results require re-requests with clarified instructions. What's the most effective way to improve reliability?

A. Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent

B. Keep the single tool but add an analyze_type enum parameter requiring explicit selection between extraction, summarization, and verification modes

C. Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats

D. Split the generic tool into purpose-specific tools – extract_data_points, summarize_content, verify_claim_against_source – each with defined input/output contracts

**Correct Answer: D**

Previous Questions

Next Questions

Browse atleast **50%** to increase passing rate

Viewing page 3 out of 14 pages.

Viewing questions **21-30** out of 134 questions